Instituto Politécnico Nacional

Escuela Superior de Cómputo

Tecnologías para el desarrollo web

Reporte de Práctica: Ejercicio API Frases

Profesor: Dr. José Asunción Enríquez Zárate

Alumno: Mauro Alfredo Enciso Galaz

maualf.enciso@gmail.com

4BM2

Fecha de entrega: 20 de octubre de 2026

Índice

1 Introducción **2**
1.1 Objetivos de la Actividad . . . 2

2 Conceptos **3**
2.1 API RESTful . . . 3
2.2 Patrón CRUD . . . 3
2.3 JPA e Hibernate . . . 3

3 Desarrollo **4**
3.1 *Configuración de la Base de Datos (application.properties)* . . . 4
3.2 *Capa de Control REST (FraseController.java)* . . . 5

4 Resultados: Pruebas con Insomnia **6**
4.1 Prueba de Insomnia . . . 6
4.2 Prueba metodo POST . . . 6
4.3 GET poblado . . . 7
4.4 GET por ID . . . 7
4.5 Editar Frase . . . 8
4.6 DELETE por ID . . . 8

5 Conclusión **10**

6 Referencias Bibliográficas **11**

Índice de figuras

1 Evidencia de la operación GET vacio 6
2 Evidencia de la operación POST 6
3 Evidencia de la operación GET 7
4 Evidencia de la operación GET por ID 7
5 Evidencia de la operación editar usando POST 8
6 Evidencia de la operación eliminar usando DELETE 8
7 Evidencia de eliminado usando GET 9

Índice de cuadros

1 Resumen de Endpoints y Respuestas HTTP del Servicio RESTful 10

1 Introducción

El desarrollo de aplicaciones empresariales modernas se fundamenta en la arquitectura cliente-servidor y la exposición de servicios mediante Interfaces de Programación de Aplicaciones (API) de tipo RESTful. Java se ha consolidado como la plataforma de software dominante para el desarrollo de la capa de procesamiento backend gracias a su robustez, portabilidad y estricto paradigma de Programación Orientada a Objetos.

En el contexto de la materia Tecnologías para el Desarrollo de Aplicaciones Web, este reporte documenta de manera metódica el diseño, implementación y validación de un servicio web backend con arquitectura RESTful capaz de gestionar las operaciones fundamentales de un CRUD (Create, Read, ReadAll, Update, Delete) sobre un dominio de entidades persistentes alojadas en una base de datos relacional MySQL.

1.1 Objetivos de la Actividad

- Crear una API con soporte completo a las operaciones típicas de un CRUD.
- Conectar la API de manera persistente con una base de datos relacional (MySQL).
- Utilizar clientes de pruebas HTTP como Insomnia para interactuar con la API expuesta y validar las respuestas HTTP.

2 Conceptos

Para garantizar la comprensión del proyecto, a continuación se detallan los conceptos teóricos fundamentales aplicados en la práctica.

2.1 API RESTful

Estilo arquitectónico para servicios web basados en recursos mapeados mediante URIs y manipulados utilizando verbos estándar del protocolo HTTP (GET, POST, PUT, DELETE).

2.2 Patrón CRUD

Conjunto de las cuatro funciones básicas de la persistencia de datos:

- **Create:** Inserción de un nuevo recurso en la base de datos (POST).
- **Read:** Consulta de un recurso específico identificado por su clave primaria (GET por ID).
- **ReadAll:** Consulta del catálogo completo de recursos (GET general).
- **Update:** Modificación de los atributos de un recurso existente (PUT o POST por ID).
- **Delete:** Eliminación de un recurso persistente (DELETE por ID).

2.3 JPA e Hibernate

Java Persistence API (JPA) añade un mapeo objeto-relacional (ORM) que traduce las clases y objetos de Java en tablas y tuplas SQL de manera automática, abstrayendo la complejidad de las consultas manuales de JDBC.

3 Desarrollo

El desarrollo del proyecto se estructuró siguiendo el patrón de diseño por capas de Spring Boot, garantizando una separación entre la interfaz HTTP y la capa de acceso a datos MySQL.

3.1 *Configuración de la Base de Datos (application.properties)*

El archivo de configuración enlaza el motor Spring Data JPA con la instancia MySQL local. Se establecen las credenciales, el driver y las estrategias de nombrado físico para respetar el camelCase en las tablas.

application.properties

```
spring.application.name=Frases4bm2

spring.datasource.url=jdbc:mysql://localhost:3306/frases
spring.datasource.username=root
spring.datasource.password=root
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA / Hibernate Configuration
spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
spring.jpa.hibernate.ddl-auto=none

# Conservar exactamente los mismos nombres de campos y tablas de Java en MySQL (camelCase)
spring.jpa.hibernate.naming.physical-strategy=
org.hibernate.boot.model.naming.PhysicalNamingStrategyStandardImpl
spring.jpa.hibernate.naming.implicit-strategy=
org.hibernate.boot.model.naming.ImplicitNamingStrategyJpaCompliantImpl
```

3.2 *Capa de Control REST (FraseController.java)*

El controlador mapea las peticiones HTTP externas dirigidas a la ruta base `/api/frase` y gestiona los métodos del CRUD solicitando el procesamiento a la capa de Servicio.

FraseController.java

```java
package com.ipn.mx.frases.control;

import com.ipn.mx.frases.domain.Frase;
import com.ipn.mx.frases.service.FraseService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/frase")
public class FraseController {

    @Autowired
    private FraseService service;

    @GetMapping("/frases")
    public List<Frase> readAll(){
        return service.findAll();
    }

    @GetMapping("/frases/{id}")
    public Frase read(@PathVariable Long id){
        return service.findById(id);
    }

    @PostMapping("/frases")
    @ResponseStatus(HttpStatus.CREATED)
    public Frase save(@RequestBody Frase frase){
        return service.save(frase);
    }

    @PostMapping("/frases/{id}")
    @ResponseStatus(HttpStatus.OK)
    public Frase update(@RequestBody Frase frase, @PathVariable Long id){
        Frase f = service.findById(id);
        if (f != null) {
            f.setTextoFrase(frase.getTextoFrase());
            f.setAutorFrase(frase.getAutorFrase());
            f.setOrigenFrase(frase.getOrigenFrase());
            return service.save(f);
        }
        return null;
    }

    @DeleteMapping("/frases/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id){
        service.deleteById(id);
    }
}
```

4 Resultados: Pruebas con Insomnia

A continuación se presentan las evidencias de pruebas realizadas utilizando el cliente de REST Insomnia, validando cada una de las operaciones del CRUD hacia la base de datos MySQL.

4.1 Prueba de Insomnia

Primero, envío una petición HTTP GET a `http://localhost:8080/api/frase/frases` para que me regrese un arreglo vacio (`[]`) y así saber si la API esta bien conectada y la base de datos vacia.

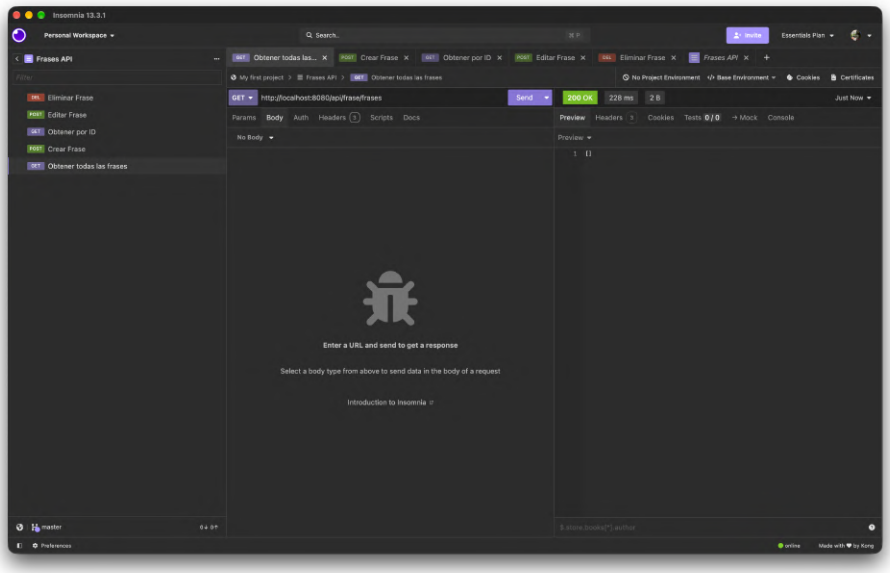


Figura 1: Evidencia de la operación GET vacio

4.2 Prueba metodo POST

Empezamos a poblar la base de datos con el metodo post, agregando frases una por una para asi hacer la prueba del metodo post.

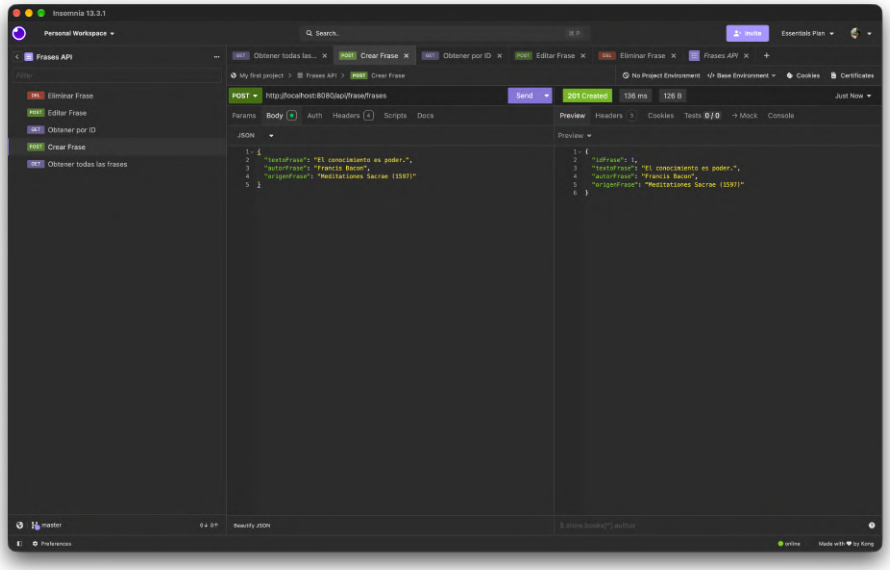


Figura 2: Evidencia de la operación POST

4.3 GET poblado

Después de ingresar varias frases, podemos ver que la base de datos esta poblada.

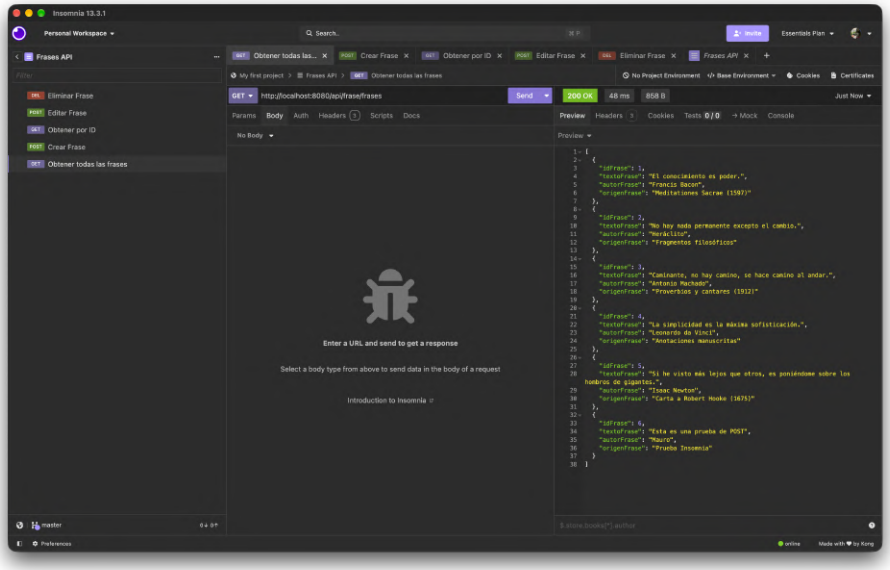


Figura 3: Evidencia de la operación GET

4.4 GET por ID

Para obtener unicamente un dato, podemos aprovechar la funcion `getById()` de nuestro controlador, escribiendo en la url el ID de la frase que queremos obtener: `http://localhost:8080/api/frase/frases/3`.

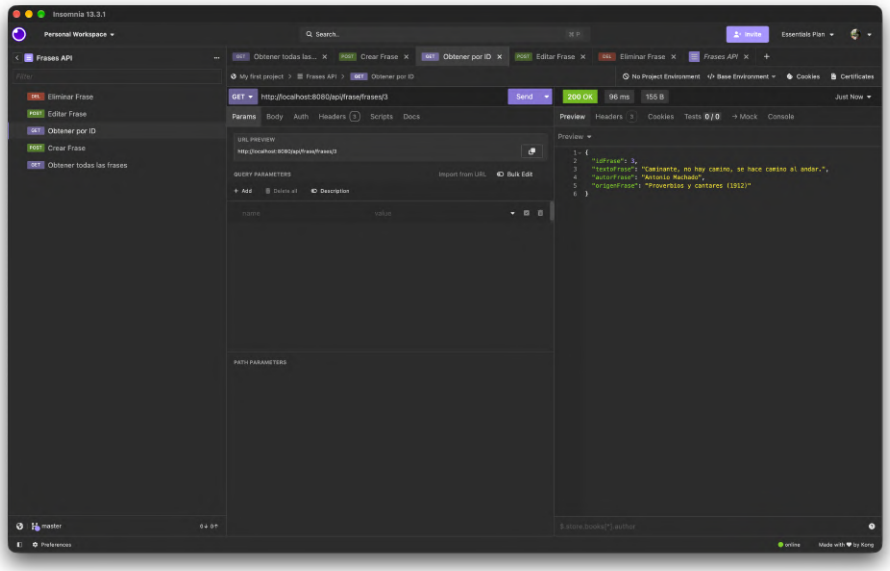


Figura 4: Evidencia de la operación GET por ID

4.5 Editar Frase

Para editar una frase en particular, utilizamos el metodo POST y la url con el ID de la frase a modificar, en este caso se actualizara la frase con ID=5: `http://localhost:8080/api/frase/frases/5`.

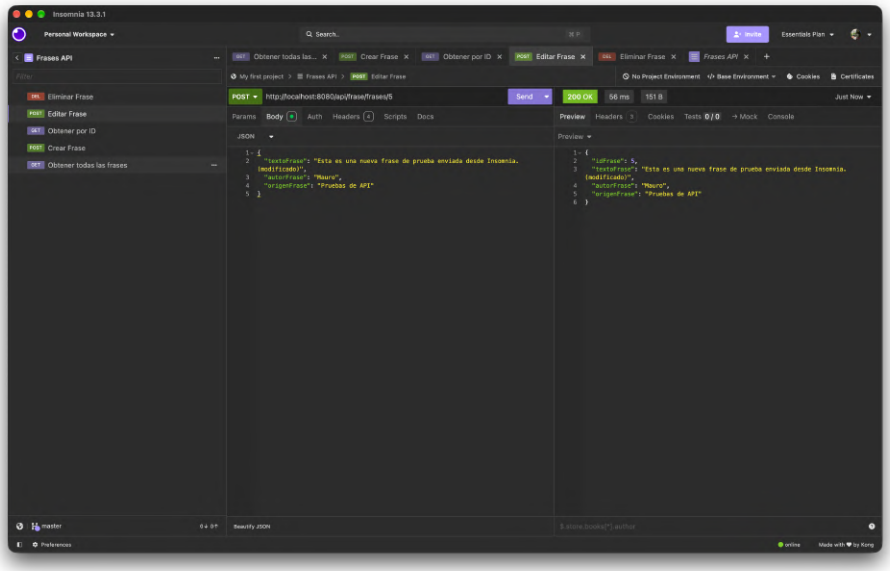


Figura 5: Evidencia de la operación editar usando POST

4.6 DELETE por ID

Para eliminar una frase en especifico, utilizamos el metodo DELETE y la url con el ID de la frase a eliminar, en este caso se borrará la frase con ID=5: `http://localhost:8080/api/frase/frases/5`.

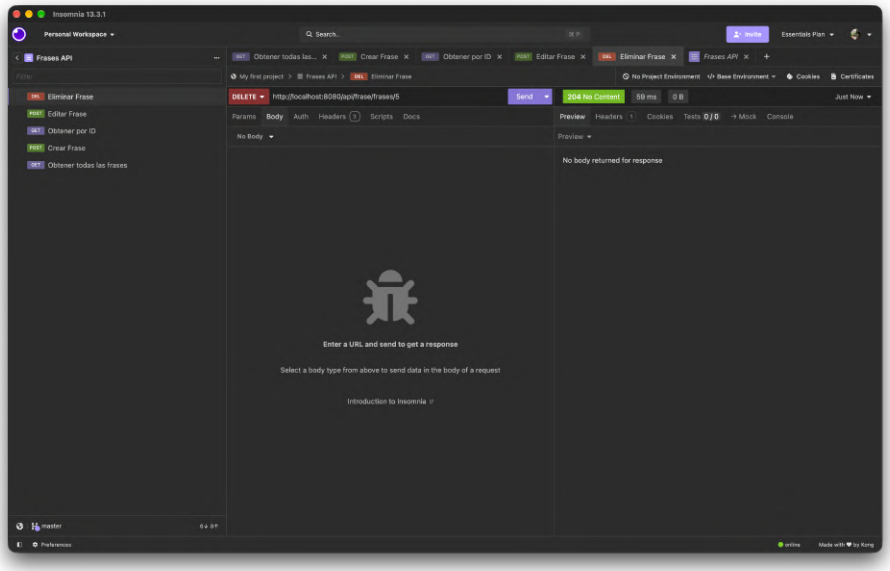


Figura 6: Evidencia de la operación eliminar usando DELETE

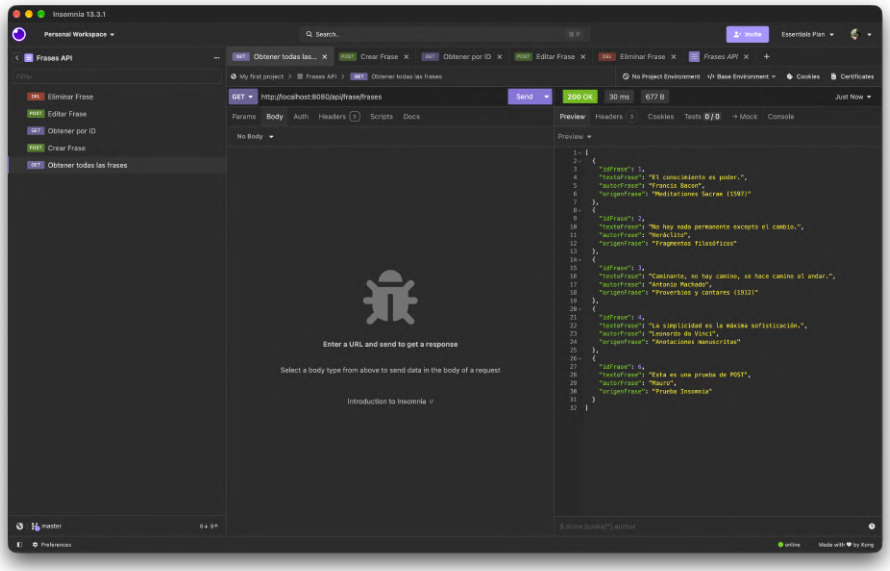


Figura 7: Evidencia de eliminado usando GET

5 Conclusión

La realización de esta práctica permitió comprender e implementar de forma integral la conexión entre una aplicación Java backend y un sistema de gestión de bases de datos relacionales (MySQL) mediante la construcción de un servicio RESTful utilizando el framework Spring Boot.

Además, la validación metódica de cada salida mediante la herramienta Insomnia confirmó el cumplimiento de la especificación HTTP para servicios web REST, asociando adecuadamente los códigos de estado HTTP a las operaciones correspondientes del ciclo de vida CRUD.

Operación CRUD	**Método HTTP**	**Código HTTP Retornado**
Create	POST	201 CREATED
Read (Individual)	GET	200 OK
ReadAll (Colección)	GET	200 OK
Update	POST / PUT	200 OK
Delete	DELETE	204 NO CONTENT

Cuadro 1: Resumen de Endpoints y Respuestas HTTP del Servicio RESTful

Mauro Alfredo Enciso Galaz

6 Referencias Bibliográficas

Referencias

[Cosmina, 2018] Iuliana Cosmina. *Java for Absolute Beginners: Learn to Program the Fundamentals the Java 9+ Way* Apress, 2018.

[Deitel, 2012] Paul Deitel, Harvey Deitel. *Java How to Program (9.a ed.)* Prentice Hall, 2012.

[Oracle, 2023] Oracle Corporation. *Java Database Connectivity (JDBC) Developer Guide and API Reference* Oracle Release Notes, 2023.

[Varga, 2019] M. Varga, A. Zoric. *Architectural Patterns for Persistent Relational Storage in Modern Java RESTful Web Services* Bielefeld Academic Search Engine (BASE), 2019.